

A Pretreatment Workflow Scheduling Approach for Big Data Applications in Multicloud Environments

Bing Lin, Wenzhong Guo, Naixue Xiong, *Senior Member, IEEE*, Guolong Chen, Athanasios V. Vasilakos, and Hong Zhang

Abstract—The rapid development of the latest distributed computing paradigm, i.e., cloud computing, generates a highly fragmented cloud market composed of numerous cloud providers and offers tremendous parallel computing ability to handle big data problems. One of the biggest challenges in multiclouds is efficient workflow scheduling. Although the workflow scheduling problem has been studied extensively, there are still very few primal works tailored for multicloud environments. Moreover, the existing research works either fail to satisfy the quality of service (QoS) requirements, or do not consider some fundamental features of cloud computing such as heterogeneity and elasticity of computing resources. In this paper, a scheduling algorithm, which is called multiclouds partial critical paths with pretreatment (MCPCPP), for big data workflows in multiclouds is presented. This algorithm incorporates the concept of partial critical paths, and aims to minimize the execution cost of workflow while satisfying the defined deadline constraint. Our approach takes into consideration the essential characteristics of multiclouds such as the charge per time interval, various instance types from different cloud providers, as well as homogeneous intrabandwidth vs. heterogeneous inter-bandwidth. Various types of workflows are used for evaluation purpose and our experimental results show that the MCPCPP is promising.

Index Terms—Big data, cloud computing, multiclouds, scheduling, scientific workflow.

Manuscript received November 28, 2015; revised March 23, 2016; accepted April 6, 2016. Date of publication April 14, 2016; date of current version September 30, 2016. This work was supported in part by the National Natural Science Foundation of China under Grant 61571129, in part by the Program for New Century Excellent Talents in Fujian Province University under Grant JA13021, in part by the Fujian Natural Science Funds for Distinguished Young Scholar under Grant 2014J06017, and in part by the Natural Science Foundation of Fujian Province under Grant 2014J01231. This work was presented in part at the 29th IEEE International Parallel and Distributed Processing Symposium (IEEE IPDPSW 2015) with the title “Cost-Driven Scheduling for Deadline-Constrained Workflow in Multiclouds” [15]. The associate editor coordinating the review of this paper and approving it for publication was H. Lutfiyya. (Corresponding authors: Naixue Xiong and Wenzhong Guo.)

B. Lin, W. Guo, G. Chen, and H. Zhang are with the College of Mathematics and Computer Sciences, Fuzhou University, Fuzhou 350108, China, also with Fujian Provincial Key Laboratory of Network Computing and Intelligent Information Processing, Fuzhou 350108, China, and also with Fujian Collaborative Innovation Center for Big Data Applications in Governments, Fuzhou 350003, China (e-mail: wheellx@163.com; fzugwz@163.com; cgl@fzu.edu.cn; fzuzhtkl@163.com).

N. Xiong is with the School of Optical-Electrical and Computer Engineering, University of Shanghai for Science and Technology, Shanghai 200093, China, and also with the Department of Business and Computer Science, Southwestern Oklahoma State University, Weatherford, OK 73096 USA (e-mail: xionгнаixue@gmail.com).

A. V. Vasilakos is with the Department of Computer Science, Electrical, and Space Engineering, Luleå University of Technology, Skellefteå 971 87, Sweden (e-mail: th.vasilakos@gmail.com).

Digital Object Identifier 10.1109/TNSM.2016.2554143

I. INTRODUCTION

MOST applications for the Big Data problems are consisted of hundreds of interdependent tasks which are generally compute-intensive and data-intensive [1]. A popular and effective method to represent such complicated applications is to adopt the DAG structured workflow model [2]–[4], which is widely used in many Big Data scientific fields such as bioinformatics, high-energy physics, and astronomy. These workflows are usually depicted as a set of tasks interconnected via control/data dependencies. Workflow-based researches have been studied for many years in the environments of HPC clusters and grids. However, with the emergence of novel distributed computing paradigm, i.e., cloud computing, new technical approaches are required to address the new challenges and constraints arising from the cloud environment.

Over the years, the distributed computing platforms have evolved from the shared resource to the utility-based environments [5]. This technology enables end users to eliminate local IT and move their business to the remote clouds, to use the specific services under certain QoS requirements via some defined Service Level Agreements (SLAs) in a pay-as-you-go manner [6]. There are a number of different commercial cloud providers such as GoGrid [7], Amazon EC2 [8] and Rackspace [9], which are delivering various technical solutions and services to remote clients. The cloud services can be classified into a group of as-a-service terms, namely Infrastructure as a Service (IaaS), Software as a Service (SaaS), and Platform as a Service (PaaS). This paper concentrates on IaaS clouds, which offer a pool of virtual, heterogeneous, and unlimited resources that can be obtained on demand to the end users. Furthermore, they are flexible to acquire or release resources with various configurations to best meet applications’ needs. Despite of the flexibility of different services, efficient and innovative scheduling strategies for workflow jobs need to be designed and implemented for efficient resources utilization.

Different cloud providers offer a number of different virtual machine (VM) instance types. For example, the price charging policies (e.g. reserved, on-demand or spot), the paying time interval, and the price for different instance types, etc, which make it hard for the end users to select the suitable instances for deploying their applications. Moreover, the faster intra-bandwidth in a single cloud and the slower inter-bandwidth between two different clouds have a significant impact on the deadline-constrained workflows arrangement across different cloud providers. Due to the complicated model of Multiclouds such as the diversities of different cloud providers’

resources, the homogeneous intra-bandwidth vs. heterogeneous inter-bandwidth, and the pricing model of the different clouds, it urgently needs an excellent scheduling technique that can not only improve service quality in terms of the QoS requirements but also reduce the cost of end users. Although many works have addressed the scheduling problem in traditional distributed systems such as grids, very few works investigate this issue across multiple clouds.

Workflow scheduling is a problem of mapping each task in a workflow to the appropriate VMs and sorting the tasks on each specific VM to meet some given performance metrics. The tremendous parallel computing ability of cloud computing encourages investigators to find out its drawbacks and advantages on scheduling Big Data scientific workflows in the cloud environments [10]. However, most of the existing research works fail to either satisfy the QoS requirements, or do not consider some fundamental principles of cloud computing such as the pay-as-you-go manner, the heterogeneity and the elasticity of computing resources. The previous works proposed a few methods for scheduling workflow on the traditional distributed systems, which mainly focused on the metric of the workflow's makespan (total execution time). However, there is another significant element other than makespan, namely economic cost in clouds. Generally, slower services are cheaper than faster services [5], so the scheduler needs to achieve a tradeoff between the cost and runtime. It is difficult to address the cost-driven scheduling for the complicated deadline-constrained Big Data workflow.

In this paper, we mainly investigate the deadline-constrained workflow scheduling strategy for the big data applications in Multi-clouds. Our work is based on heuristic optimization approach, namely Partial Critical Paths (PCP). PCP is a traditional task-driven technique and closely related to the structure of task graphs [11]. It focuses on finding all of the critical paths based on the established guidelines, and then processing each path with some specific manners respectively to get an optimal result. Many Big Data problems from different domains have been successfully addressed by adapting PCP approach. For instance this approach has been applied to solve the problems in the areas of video coding [12], VLSI design [13], and healthcare systems [14].

In our previous work [15], we adopted the PCPs approach for generating a Big Data workflow scheduling strategy in Multi-clouds, which is called Multi-Cloud Partial Critical Paths (MCPCP). Compared with [15], the key contributions of this study are summarized as follows:

- We enrich our strategy with a new scheduling model, which considers more essential characteristics for scheduling workflow in Multi-clouds such as the boot and shutdown time of instances, the different inter-bandwidth and the data transfer cost between two different providers.
- For the sake of compressing the amount of data between two adjacent tasks and reducing the runtime of our proposed strategy, we merge the two tasks with a common 'directed cut-edges' into a single task based on the workflow structure. It has significant improvement on the performance of our scheduling result.
- In order to adapt to the new price scheme in current cloud market and reduce the execution cost of the workflow, we

enhance the process of 'best-fit instance' selection, which can finish each task of the corresponding PCP before its latest finish time.

- A scheduling strategy, considering the workflow's structure and the characteristics of Multi-clouds, is proposed to minimize the workflow's execution cost while satisfying its deadline constraint in Multi-clouds.

The rest of this paper is organized as follows: in Section II, the related work is presented. The scheduling system model is described in details in section III, and then the basic definitions and the proposed MCPCPP algorithm are presented in section IV. In section V, comparison of our algorithm with other three state-of-the-art algorithms can be found. Finally, concluding remarks are made in section VI.

II. RELATED WORK

A. Traditional Distributed Environment

Over the years, scientific workflow scheduling in parallel distributed systems such as multiprocessing has been studied extensively and proved to be a NP-hard problem [16]. It means that it is impossible to produce an optimal result within polynomial time unless $P = NP$. Thus many heuristic algorithms were proposed to generate near-optimal or approximate solutions to satisfy the QoS requirements of the end users. A large range of the previous presented solutions were based on the shared community environments such as community grids. Assuming that limited computation or storage resources were available, in general, the scheduling objective was to minimize the workflow's execution time, where the execution cost of the workflow was rarely focused on. Chen and Zhang [17] proposed a Big Data workflow scheduling algorithm based on Ant Colony Optimization (ACO), which aimed to find a solution that optimized the user-preferred QoS parameters and satisfied all user QoS constraints in grids. Hiraes-Carbajal *et al.* [18] elaborated new workflow strategies with user runtime estimations in a grid. Their approaches had a better performance in terms of critical path slowdown, average critical path waiting time, and approximation factor. Gao *et al.* [19] presented a novel workflow scheduling heuristic based on the group scheduling and list scheduling, which improved resource utilization and decreased total execution time in grids. Yu and Buyya [20] developed a new type of genetic algorithm to accomplish a budget-constrained Big Data workflow scheduling in utility grids, which minimized the total execution time while meeting a specified budget.

Other works also adopted PCPs to address the Big Data workflow scheduling issue. A QoS-based algorithm for scheduling a single workflow was presented for the utility grids [21], which depended on the PCPs of the workflow. It tried to minimize the total execution cost of the workflow through distributing the specified deadline over all tasks in a partial critical path recursively, and scheduled each task to the cheapest service which could satisfy the tasks' sub-deadline. Moreover, it provided some suggestions for scheduling workflow with PCPs method. This workflow scheduling algorithm focused on the grid environments, however, it had many significant differences compared with the commercial clouds.

B. Cloud Environment

The above-mentioned works offered a significant insight into the potential solutions and challenges for the workflow scheduling. Nevertheless, they were not suitable for the utility-based environments such as IaaS clouds. The cloud environments have various specific characteristics to be considered when developing a workflow scheduling algorithm. Malawski *et al.* [22] addressed an important issue concerning the efficient management of Big Data workflow ensembles under deadline and budget constraints in IaaS clouds. They elaborated various static and dynamic algorithms for resource provisioning and task scheduling, which aimed to maximize the amount of completed workflows while meeting the defined QoS requirements. They acknowledged that there were different delays when dealing with the leased instances from cloud providers such as the termination and acquisition delays. However, their solution only considered a single type of instance, which ignored the heterogeneity of cloud resources.

Another work, scheduling Big Data workflow ensembles in clouds, was presented by Mao and Humphrey [23]. They acknowledged that the cloud providers offered various sizes/costs instances, which could be leased on demand based on the application's performance requirements. In addition, they proposed a dynamic approach for minimizing the execution cost of workflows according to the cloud's pricing model, i.e., instances were paid based on a time interval, which generally was one hour. In order to further reduce the execution cost, they adopted a set of heuristics such as bundling adjacent tasks that preferred the same instance type into a single one, arranging the most cost-efficient instance for each task, and consolidating idle time slot for all the running instances. This work could reduce the workflows' execution cost, but it ignored the different bandwidths among various clouds.

More in line with the work done in this paper was the solution proposed by Rodriguez and Buyya [5], which presented a static approach based on the Particle Swarm Optimization (PSO) technique for scheduling a Big Data workflow in cloud platforms. Their algorithm considered the basic cloud features such as leased instances delays, instance heterogeneity, as well as time interval pricing and pay-as-you-go manner. They minimized the overall workflow execution cost through a global optimization technique, which met the specified deadline requirements. However, they randomly mapped the tasks to the corresponding instances, and failed to make use of the characteristics and structure of the workflow to produce a better result. Furthermore, their optimization technique only considered scheduling workflow in a single cloud.

The Multi-cloud computing research domain was relatively budding with very few works available. The concept of Multi-cloud computing was first introduced by Keahey *et al.* [24], which built a virtual cluster and created a trusted networking distributed in Multi-clouds. Buyya *et al.* elaborated the important architectural elements and challenges for utility-oriented federation of the Multi-cloud environments [25]. Fard *et al.* [26] proposed a truthful mechanism for scheduling Big Data workflows, considering two objectives: monetary cost and makespan, in a commercial Multi-cloud environment. They

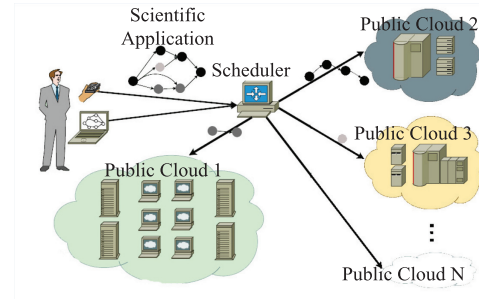


Fig. 1. System architecture for our scheduling paradigm.

tailored their approach to be suitable for scheduling the workflow applications dynamically. Their work took advantage of the elasticity of cloud resources, but ignored the heterogeneous nature of cloud and considered only a single instance type in a specified cloud. Furthermore, they aimed to optimize both makespan and cost, which focused on the workflow's execution cost while meeting the specified deadlines. Li *et al.* [27] proposed two online optimization algorithms for scheduling preemptable tasks across multiple clouds, which did not consider the data transfer cost and the elasticity of cloud resources.

Abrishami *et al.* [28] improved their previous research work [21] in utility grids and proposed two workflow scheduling approaches with a polynomial time complexity for the cloud platforms. They adapted the PCPs technique and attempted to minimize the workflow's execution cost while satisfying the user-defined deadline, whose topic was similar to our work. However, they achieved their goals only in a single cloud and did not consider the different bandwidths between various providers. In addition, they ignored the delays when dealing with the leased instances from cloud providers.

III. SCHEDULING SYSTEM MODEL

The system architecture for our scheduling paradigm is shown in Fig. 1, which mainly composed of a Big Data workflow application model, an IaaS Multi-cloud model, and a scheduler with performance criterions.

A Big Data workflow is represented by a directed acyclic graph $W = (Vertex, Edge)$, where *Vertex* is a finite set of n task nodes $\{t_1, t_2, \dots, t_n\}$, and *Edge* is defined as a set of data dependencies between tasks. Each data dependency edge $e_{ij} = (t_i, t_j)$ indicates the dependent relationship between task t_i and task t_j , in which t_i is the direct precursor of t_j and t_j is the direct successor of t_i . A task cannot be executed until all its precursors have been completed. In a given Big Data workflow graph, a task with no direct precursors is called enter-task, and the exit-task has no direct successors. As our scheduling algorithm requires a single-entry and single-exit task graph, we add a zero-cost pseudo enter-task (exit-task) to the beginning (end) of the workflow. The actual enter-tasks (exit-tasks) are connected to the pseudo enter-task (exit-task) with zero-weight data dependencies, which has no effect on the schedule. Fig. 2 shows a Big Data workflow sample added with a pseudo enter-task t_s and a pseudo exit-task t_e . Each node represents a task in the workflow and the arcs show the data transfer times

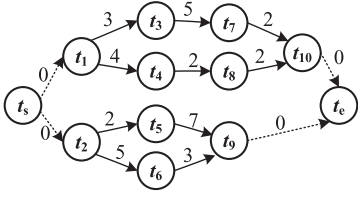


Fig. 2. Big Data workflow sample.

between two tasks. There are more than one actual enter-tasks (i.e., t_1 and t_2) and exit-tasks (i.e., t_9 and t_{10}) in the workflow sample. Besides, each workflow application W has a corresponding deadline, $D(w)$, before which the workflow W must be completed.

Our Multi-cloud model consists of many different IaaS cloud providers $P = \{p, q, \dots, r\}$ which offer diverse virtualized resources to the end users. In particular, we assume that each provider can provide a set of computation services to run the tasks of workflow and storage services which are affiliated to the corresponding computation services as local storage to afford enough space for in-out files like Amazon EC2 and EBS, respectively. Furthermore, each provider p offers a set of computation services $S_p = \{s_{p1}, s_{p2}, \dots, s_{pm}\}$ with diverse parameters such as amount and speed of CPUs, memory size and price. Usually, lower QoS parameters, e.g. less memories or slower CPUs, mean a cheaper price. In this paper, we mainly focus on the computation service and there is no limitation on employing the service, i.e., the end user can lease any number of instances from any computation service of each provider at any time, which accords with the illusion of unlimited cloud resources [10]. In addition, a new leased instance vm_{pi} provided by cloud p requires a specific initial boot time $T_{boot}(vm_{pi})$ to be available to the end user and properly initialized; this time needs to be considered as it could have a significant impact on the schedule generation [29]. Also, we assume that an instance running a task t_i should keep active until all the output data is transmitted to the instances executing the direct child tasks of t_i [5].

Furthermore, the execution time of task t_i on computation service s_{pj} is represented as $T_{exe}(t_i, s_{pj})$. Each service on different cloud provider has different price/performance ratio for each task [30]. We assume that all computation services and storage services of a cloud provider are kept in the same region and different providers' services are stored far from each other, so that the intra-bandwidth in a single cloud provider is much faster than the inter-bandwidth between two different cloud providers. $B_{intra}(s_{pi}, s_{pj})$ represents the intra-bandwidth of service s_{pi} to service s_{pj} on provider p , and $B_{inter}(s_{pi}, s_{qj})$ is the inter-bandwidth between service s_{pi} of provider p and service s_{qj} of provider q , respectively. The intra-data transfer time with $B_{intra}(s_{pi}, s_{pj})$, $TT_{intra}(e_{ij}, s_{pi}, s_{pj})$, and inter-data transfer time with $B_{inter}(s_{pi}, s_{qj})$, $TT_{inter}(e_{ij}, s_{pi}, s_{qj})$, of a data dependency e_{ij} are described as follows:

$$TT_{intra}(e_{ij}, s_{pi}, s_{pj}) = \frac{Data(e_{ij})}{B_{intra}(s_{pi}, s_{pj})} \quad (1)$$

$$TT_{inter}(e_{ij}, s_{pi}, s_{qj}) = \frac{Data(e_{ij})}{B_{inter}(s_{pi}, s_{qj})} \quad (2)$$

where the amount of data in a dependency e_{ij} , $Data(e_{ij})$, is the data amount transferred from task t_i to task t_j before task t_j starts. If both corresponding tasks t_i and t_j are scheduled on the same instance of a service, the cost of $TT_{intra}(e_{ij}, s_{pi}, s_{pj})$ becomes 0. Generally, a cloud provider allocates its services in the same area, so we assume that the $B_{intra}(s_{pi}, s_{pj})$ is roughly equal to a fixed value for each provider p . With this assumption, $TT_{intra}(e_{ij}, s_{pi}, s_{pj})$ and $TT_{inter}(e_{ij}, s_{pi}, s_{qj})$ are independent of the services executing on them, but only depend on the selected provider and the amount of transferring data.

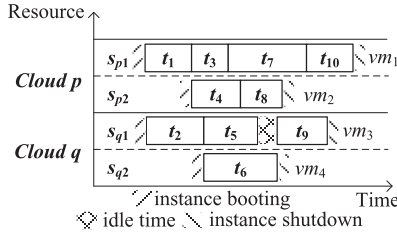
The price model is based in a pay-as-you-go manner with on-demand mechanism and each cloud provider p charges users on a basic unit of time λ_p . For example, for $\lambda_p = 60$ minutes, it indicates that the cost for running a task on a instance for 61 minutes is same as the cost for 2 hours in the same condition. We assume that each computation service s_{pi} has a corresponding cost c_{pi} for each time interval. We do not consider the fee of data transfer in a single cloud as many IaaS providers ignore the charge if the data are made in the same datacenter. Nevertheless, we do consider the data transfer cost between two different providers. $c_{p,q}$ represents the data transfer cost per GB from provider p to provider q . At present, we do not take the cost for extra services into account, such as data persistent storage, automatic load balancing and resource monitoring, as these services cost are negligible compared with the tasks computation [31].

The scheduler schedules the Big Data workflow in Multi-clouds with a specific performance criterion, which is to minimize the economic cost of workflow execution while completing it before the user-defined deadline. A schedule $S = (Re, Map, Ttotal, Ctotal)$ is defined in terms of a group of resources from different providers, a task to resource mapping, the total execution time and the total execution cost including computation and data transmission for a Big Data workflow. Fig. 3 exhibits a schedule for the workflow shown in Fig. 2, and the resources are offered by provider p and provider q . $Re = \{vm_1, vm_2, \dots, vm_r\}$ represents the set of instances which needs to be leased; each instance vm_i has a corresponding computation service type $Type(vm_i)$ as well as the leased start time $Tls(vm_i)$ and leased end time $Tle(vm_i)$. Map is the mapping for each workflow task; $map(t_i, vm_j)$ indicates that task t_i is scheduled on the instance vm_j , and each task t_i has a actual start time $AST(t_i)$ and actual end time $AET(t_i)$ associated to it after scheduling it on a instance. Therefore, the total execution time and the total execution cost are calculated as formula (3) and formula (4), respectively:

$$Ttotal = \max_{t_i \in W} \{AET(t_i)\} \quad (3)$$

$$Ctotal = \sum_{i=1}^{|Re|} c_{pj} \cdot \left[\frac{Tle(vm_i) - Tls(vm_i)}{\lambda_p} \right] + \sum_{i=1}^n \sum_{k(i)} c_{p,q} \cdot Data(e_{ij}) \cdot s_{k(i)} \quad (4)$$

where c_{pj} and λ_p are both variables based on the computation service type $Type(vm_i)$, which is equal to s_{pj} . In addition, $k(i)$ is the number of edges in which t_i is the parent task, and $s_{k(i)}$ is 0 whenever t_i and t_j execute on the same cloud or 1 otherwise.

Fig. 3. A schedule for the workflow shown in Fig. 2 between cloud p and q .TABLE I
SYMBOLS DEFINITION

Symbol	Definition
W	A Big Data workflow application
n	The number of the task nodes in W
t_i	Task i in W
e_{ij}	Data precedence between t_i and t_j , which indicates that t_j can not start until t_i have been completed
$D(w)$	The corresponding deadline of W
vm_i	A instance (virtual machine) i
s_{pi}	Computation service i offered by provider p
λ_p	Time interval, per which provider p charges users
c_{pi}	Execution cost of s_{pi} per λ_p
$c_{p,q}$	The data transfer cost per GB from provider p to provider q
$T_{exe}(t_i, s_{pj})$	Execution time of task t_i on s_{pj}
$B_{intra}(s_{pi}, s_{pj})$	Intra-bandwidth of s_{pi} to s_{pj} on provider p
$B_{inter}(s_{pi}, s_{qj})$	Inter-bandwidth between s_{pi} of provider p and s_{qj} of provider q
$Data(e_{ij})$	The amount of data transferred in e_{ij} ,
$AST(t_i)$	The actual start time of t_i
$AET(t_i)$	The actual end time of t_i

$c_{p,q}$ is a variable and equal to the corresponding data transfer unit price between the two different clouds when $s_{k(i)}$ is 1.

Therefore, the scheduling problem described in this paper can be defined as follows: find a schedule S with the minimum C_{total} in Multi-clouds, for which the T_{total} is less than the user-defined deadline of the Big Data workflow. This single-objective optimization problem with constraints can be represented as formula (5):

$$\begin{aligned} & \text{Minimize } C_{total} \\ & \text{subject to } T_{total} \leq D(w) \end{aligned} \quad (5)$$

Table I defines the symbols used in this paper.

IV. A MULTICLOUDS PARTIAL CRITICAL PATHS WITH PRETREATMENT STRATEGY

In this section, we first define some basic definitions associated with our Big Data workflow scheduling strategy, and then elaborate on the detail of the core algorithm MCPCPP used in the scheduler to schedule workflow in Multi-clouds with the specific performance criterion discussed before.

A. Basic Definitions

In this paper, a task assigned to an instance, is called a scheduled task, otherwise called an unscheduled task. The earliest

finish time of an unscheduled task t_i , $EFT(t_i)$, is represented by formula (6):

$$\begin{cases} EFT(t_i) = \min \{ \min_{p \in P} \{ T_{exe}(t_i, s_{pj}) + T_{boot}(vm_{pj}) \} \} \\ t_i \in \text{actual enter-tasks} \\ EFT(t_i) = \min \{ \max_{p \in P} \{ EFT(t_j) + MET(t_i, p) + TE(e_{ji}, p) \} \} \\ t_i \notin \text{actual enter-tasks} \end{cases} \quad (6)$$

$$MET(t_i, p) = \min_{s_{pj} \in S_p} \{ T_{exe}(t_i, s_{pj}) \} \quad (7)$$

$$EFT(t_{pseudoenter-task}) = 0 \quad (8)$$

where the minimum execution time of an unscheduled task t_i in cloud p , $MET(t_i, p)$, is shown as formula (7) which represents the minimum $T_{exe}(t_i, s_{pj})$ of task t_i in cloud p 's all available services S_p . In our proposed scheduling strategy, each task t_i has a preferred cloud, $preP_E(t_i)$, to get its earliest finish time $EFT(t_i)$ before scheduling. $TE(e_{ji}, p)$ is the data transfer time of a dependency e_{ji} between cloud p and cloud $preP_E(t_j)$. Obviously, the earliest finish time of the pseudo enter-task is zero defined as formula (8), and the $MET(t_{pseudoenter-task}, p)$ and $MET(t_{pseudoexit-task}, p)$ are both zero in any cloud.

There is another notion of the tasks' start time. The earliest start time of an unscheduled task t_i , $EST(t_i)$, is computed before scheduling the task. Therefore, the $EST(t_i)$ of each unscheduled task t_i is represented by formula (9), and the $EST(t_{pseudoenter-task})$ is also zero described as formula (10).

$$EST(t_i) = EFT(t_i) - MET(t_i, preP_E(t_i)) \quad (9)$$

$$EST(t_{pseudoenter-task}) = 0 \quad (10)$$

The calculation steps for each task's $EST(t_i)$ and $EFT(t_i)$ are different from the traditional methods which calculates the $EST(t_i)$ first, and then achieves the $EFT(t_i)$ based on the corresponding $EST(t_i)$. Due to the different bandwidth and computing capacity in Multi-clouds, a service s_{pj} , on which a task t_i can get its $EST(t_i)$, may not be the optimal service to achieved its $EFT(t_i)$. Because the QoS constraint for the scheduling strategy is to satisfy the defined deadline $D(w)$, we calculate the $EFT(t_i)$ first and the $EST(t_i)$ later.

We also define the latest start time of an unscheduled task t_i as $LST(t_i)$, before which the task should start so that the whole workflow W can be completed before its corresponding deadline $D(w)$. The $LST(t_i)$ of each unscheduled task t_i is computed as formula (11). Obviously, the latest start time of the pseudo exit-task, $LST(t_{pseudoexit-task})$, is equal to $D(w)$ represented by formula (12):

$$\begin{cases} LST(t_i) = D(w) - \min_{p \in P} \{ MET(t_i, p) \} \\ t_i \in \text{actual exit-tasks} \\ LST(t_i) = \max_{p \in P} \{ \min_{t_j \in t_i's \text{ children}} \{ LST(t_j) - MET(t_i, p) - TL(e_{ij}, p) \} \} \\ t_i \notin \text{actual exit-tasks} \end{cases} \quad (11)$$

$$LST(t_{pseudoexit-task}) = D(w) \quad (12)$$

where each task t_j has a preferred cloud, $preP_L(t_j)$, to get its latest start time $LST(t_j)$. $TL(e_{ij}, p)$ is the data transfer time of a dependency e_{ij} between cloud p and cloud $preP_L(t_j)$.

Accordingly, the latest finish time of unscheduled task t_i , $LFT(t_i)$, is defined as formula (13).

$$LFT(t_i) = LST(t_i) + MET(t_i, preP_L(t_i)) \quad (13)$$

In addition, we define the selected instance service for processing the task t_i , $SS(t_i)$, during scheduling as formula (14):

$$SS(t_i) = s_{p,j,k} \quad (14)$$

where $s_{p,j,k}$ is the k^{th} instance of service s_{pj} on provider p . Hence, the $AST(t_i)$ is determined after task t_i has been scheduled on its selected service.

The major concept of our scheduling strategy is the Partial Critical Path (PCP), which compresses the critical path from a start unscheduled task to an end unscheduled task in order to reduce the data transfer time and cost between the interdependent unscheduled tasks in this path. The partial critical path of a task t_i , $PCP(t_i)$, is defined as formula (15):

$$PCP(t_i) = \begin{cases} \emptyset, & \text{if } t_i \text{ doesn't has any unscheduled parents} \\ path(PCP(CP(t_i)), CP(t_i)), & \text{else} \end{cases} \quad (15)$$

$$CP(t_i) = \begin{cases} \emptyset, & \text{if } t_i \text{ has no unscheduled parents} \\ t_{index}, & \text{else} \end{cases} \quad (16)$$

$$index = subscript(\max_{t_j \in t_i's \text{ unscheduled parents}} \{EFT(t_j) + \min_{s_{pi} \in S_{p,j}, s_{qj} \in S_{qj}} \{TT_{inter}(e_{ji}, s_{pi}, s_{qj})\}\}) \quad (17)$$

where the critical parent of a task t_i , $CP(t_i)$, is t_i 's unscheduled parent which has the latest approximate data arrived time at t_i , which is represented by formula (16), and the function $subscript()$ represents the task number shown by formula (17). Clearly, the data transfer time is based on the selected services and bandwidth between their providers. Because Multi-clouds is a heterogeneous environment and the tasks' execution time varies from service to service, we select the sum of minimum execution time and minimum data transfer time between two different providers as a major element to approximately achieve a task's critical parent. Moreover, the function $path(way, t)$, where way is a path (maybe only contains a single task) and t is a task, represents a path that t is in front of way . Therefore, the $PCP(t_i)$ represents a path from a start unscheduled task to the critical parent of t_i , and each task (except $CP(t_i)$) has the approximate latest arrived time to its successive task on the path.

Finally, the concept of 'best-fit instance' is defined as the instance $s_{p,j,k}$ (already-existed or new-initiate) of a computation service from a cloud provider, which can finish each task of a corresponding PCP before its latest finish time and also meets the three conditions as following:

- The cost growth of $s_{p,j,k}$ for the corresponding PCP, $Cgrow(s_{p,j,k}, PCP)$, is the minimum. The cost growth of $s_{p,j,k}$ is the increased cost, including computing and data transfer cost, after executing a new PCP on this instance. The definition of $Cgrow(s_{p,j,k}, PCP)$ is described as formula (18):

$$Cgrow(s_{p,j,k}, PCP) = c_{pj} \cdot (T_2 - T_1) + Cdata \quad (18)$$

where $T_1(T_2)$ is the old (new) time intervals of the instance $s_{p,j,k}$ before (after) executing the PCP. When the $s_{p,j,k}$ has been initially started, the value of T_1 is 0 and T_2 is the total time intervals including instance booting and shutdown time accordingly. Moreover, the increased data cost, $Cdata$, is the new data transfer cost from each task in the PCP to the already scheduled tasks when they are scheduled in different providers' services.

- When there are more than two instances satisfying the condition *a*, the one with the minimum $Cdata$ is selected. It means that we would rather schedule tasks in the same region to reduce the data transfer cost from different providers.
- When there are more than two instances both meeting the condition *a* and condition *b*, we have to select the one with longer remaining time that is equal to the difference between the current time intervals and the actual leased time. An instance with larger remaining time is more likely to accommodate tasks with the same computing capacity.

The selection of 'best-fit instance' is based on the greedy strategy, which tries to reduce the data transfer cost, improve resource utilization efficiency and finally minimize the execution cost of workflow.

B. MCPCPP Algorithm

MCPCPP algorithm is a one-phase approach which has different structure to the PCPA proposed for the utility grid model [21]. It assigns sub-deadlines to the PCPs instead of independent tasks, which can compress and reduce the data transfer time along the PCPs. Furthermore, it tries to find a 'best-fit instance' to execute the entire PCP before its latest finish time, which adapt to the new price model in current commercial clouds. There are three important components in the MCPCPP algorithm. Firstly, in order to reduce the algorithm complexity and compress the data transfer time preliminarily, every two adjacent tasks with a common 'directed cut-edge' will be merged into a single task based on the workflow structure. Secondly, the partial critical paths with sub-deadline constraints will be found based on the 'critical parent' iterative mechanism. Thirdly, the 'best-fit' instances are allocated to each found partial critical path and all the tasks on the path are scheduled to its corresponding instance.

The overall structure of MCPCPP algorithm for scheduling a workflow W before its deadline $D(w)$ in Multi-clouds is shown in Algorithm I. This algorithm strives to minimize the total cost of executing the workflow while satisfying its deadline constraint. First, it calls the *preProcess* procedure in line 1, which preprocesses the workflow W through merging the 'directed cut-edges'. Then, it reconstitutes the workflow and initializes some parameters in lines 2–4. The actual start time of $t_{pseudoenter-task}$ and $t_{pseudoexit-task}$ are set to be 0 and $D(w)$, respectively, and the two pseudo tasks are labeled as the scheduled tasks in lines 5–6. The $AST(t_{exit-task})$ is set to be $D(w)$, which enforces the direct parents of $t_{pseudoexit-task}$, i.e., the

Algorithm 1. MCPCPP algorithm

```

procedure Schedule_Workflow( $W(\text{Vertex}, \text{Edge}), D(w), \{S_p\}$ )
1: call preProcess( $W(\text{Vertex}, \text{Edge})$ )
2: add  $t_{\text{pseudoenter-task}}, t_{\text{pseudoexit-task}}$  and the corresponding
   zero dependencies to the new  $W$  after pretreatment
3: confirm the available computation services from different
   providers
4: compute  $EFT(t_i), EST(t_i), LST(t_i)$  and  $LFT(t_i)$  for each task
   in the  $W$ 
5: initialize:  $AST(t_{\text{pseudoenter-task}}) \leftarrow 0,$ 
    $EFT(t_{\text{pseudoenter-task}}) \leftarrow 0, AST(t_{\text{pseudoexit-task}}) \leftarrow D(w)$ 
6: label  $t_{\text{pseudoenter-task}}$  and  $t_{\text{pseudoexit-task}}$  as scheduled
   tasks
7: call Schedule_all_Parents( $t_{\text{pseudoexit-task}}$ )
end procedure

```

Algorithm 2. merging the adjacent two tasks with a common ‘directed cut-edge’

```

procedure preProcess( $W(\text{Vertex}, \text{Edge})$ )
1: record the in-degrees and out-degrees [32] for each task
   while inputting the relevant information of  $W$ 
2: construct a father son nodes matrix  $\text{father-son}[][]$ , and find
   all the ‘directed cut-edge’
3: foreach  $e_{ij}$  in new  $\text{Edge}$  set do
4:   if  $\text{outdegree}(t_i)=1$  and  $\text{indegree}(t_j)=1$  then
5:     merge  $t_i$  and  $t_j$  into  $t_k$ , delete  $e_{ij}$ , update  $T_{\text{exe}}(t_k,$ 
        $s_{pj})$ 
6:   end if
7: end for
8: goto line 2 until there is no ‘directed cut-edge’
end procedure

```

actual exit-tasks of workflow W , to complete before the user’s deadline. Finally, it calls the *Schedule_all_Parents* procedure in line 7, which schedules all the unscheduled parents of its input task. As all tasks other than $t_{\text{pseudoenter-task}}, t_{\text{pseudoexit-task}}$ in the workflow are initialized to be unscheduled and the procedure will be called for $t_{\text{pseudoexit-task}}$ at first, then it will schedule all the tasks in the workflow.

1) *Pretreatment*: Algorithm II shows the pseudo-code of the *preProcess* procedure, which merges the adjacent two tasks with a common ‘directed cut-edge’ into a single task in W based on its structure. The definition of ‘directed cut-edge’ is different from traditional cut-edge [33]. It is a directed edge whose front and rear side both have only one node. Fig. 4(a) shows the structure of ‘directed cut-edge’, and the data transfer volume can be efficiently decreased through the pretreatment procedure. First, each task’s in-degree and out-degree are recorded when defining the W in line 1. Then, in order to reduce the time complexity for finding all the ‘directed cut-edge’, a father-son matrix [32] is constructed to determine whether an edge is a ‘directed cut-edge’ or not in line 2. Only when a father only has one son whose in-degree is 1, the edge linking the father and the son is a ‘directed cut-edge’. Next, each pair of ‘directed cut-edge’ tasks is merged into one task, whose execution time on different

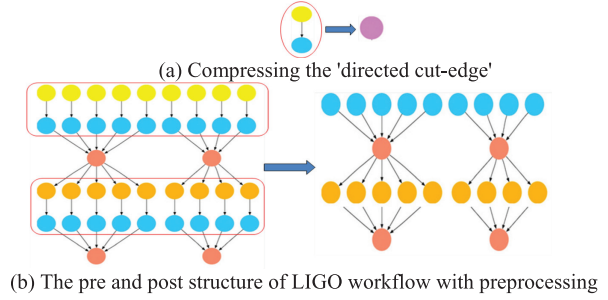


Fig. 4. Pretreatment procedure. (a) Compressing the ‘directed cut-edge’. (b) The pre and post structure of LIGO workflow with preprocessing.

Algorithm 3. Scheduling all the unscheduled parents of t_i

```

procedure Schedule_all_Parents( $t_i$ )
1: while there exists an unscheduled parent of  $t_i$  do
2:   initialize:  $t_{\text{pseudo}} \leftarrow t_i$ , stack  $Sta \leftarrow \text{null}$ 
3:   while  $t_{\text{pseudo}}$  has an unscheduled parent do
4:     push  $CP(t_{\text{pseudo}})$  into  $Sta$ 
5:      $t_{\text{pseudo}} \leftarrow CP(t_{\text{pseudo}})$ 
6:   end while
7:   initialize:  $PCP \leftarrow \text{null}$ 
8:   while  $Sta$  is not empty do
9:     insert top( $Sta$ ) to the end of  $PCP$ 
10:    pop top( $Sta$ )
11:   end while
12:   call Schedule_Path( $PCP$ )
13:   foreach  $t_i$  in the  $PCP$  do
14:     update the  $EFTs$  and  $ESTs$  for all unscheduled suc-
       cessors of  $t_i$ 
15:     update the  $LSTs$  and  $LFTs$  for all unscheduled pre-
       decessors of  $t_i$ 
16:     call Schedule_all_Parents( $t_i$ )
17:   end for
18: end while
end procedure

```

computation services from various providers is updated in lines 3–7. Finally, the merging operation continues in the new generated workflow W' iteratively until there is no ‘directed cut-edge’ in line 8. This pretreatment procedure can decrease the number of tasks, especially for the workflow containing many ‘directed cut-edges’ such as Epigenomics workflow [34], thus the algorithm complexity and data transfer edges can be reduced. The pre and post structure of LIGO workflow with preprocessing is described in Fig. 4(b).

2) *Scheduling all the Unscheduled Parents of Task t_i* : Algorithm III shows the pseudo-code of the *Schedule_all_Parents* procedure, which receives an already-scheduled task as input and schedules all of its unscheduled parents before its actual start time. Firstly, *Schedule_all_Parents* finds the PCP of the input task in lines 3–11. When this procedure is first called, it starts with input $t_{\text{pseudoexit-task}}$ and follows back the corresponding critical parents until it arrives at $t_{\text{pseudoenter-task}}$. Therefore, the overall actual critical path of the workflow is found in the first

Algorithm 4. Scheduling the entire path of PCP**procedure** *Schedule_Path(PCP)*

```

1: foreach applicative existing instance do
2:   find an instance  $s_{p,j,k}$  for the PCP with minimum
      $C_{grow}(s_{p,j,k}, PCP)$ 
3: end for
4: if  $s_{p,j,k}$  is not exist then
5:   initiate a new instances  $s_{p,j,k}$  with minimum  $C_{grow}(s_{p,j,k},$ 
      $PCP)$ , which can finish each task of the PCP before its
      $LFT$ 
6: end if
7: schedule the PCP on the instance  $s_{p,j,k}$ , which is called
   'best-fit instance'
8: foreach  $t_i$  in the PCP do
9:    $SS(t_i) \leftarrow s_{p,j,k}$ 
10:  calculate  $AST(t_i), AET(t_i)$ 
11:  set  $t_i$  as a scheduled task
12: end for
end procedure

```

call of this algorithm. Then the *Schedule_Path* procedure is called in line 12, which receives a PCP as input and schedules the entire path on a 'best-fit instance' to execute all the tasks in the PCP before its latest finish time. The detail description of the *Schedule_Path* procedure is discussed in next section. After a task in the PCP is scheduled, its actual start time and actual finish time is determined, which may affect the *LSTs* and *LFTs* of its predecessors and the *EFTs* and *ESTs* of its successors. Therefore, these parameters related to all the tasks in the PCP are updated in lines 14-15. Finally, the algorithm begins to schedule all the parents of each task in the PCP by calling *Schedule_all_Parents* recursively which is from beginning to end of the PCP in line 16.

3) *Scheduling Partial Critical Path:* The pseudo-code for scheduling the entire path of a PCP on a 'best-fit instance' is shown in Algorithm IV, which receives a path as input. Firstly, the procedure adopts the greedy strategy and tries to find the minimum cost growth instance to schedule the path from all *applicative* existing instances in lines 1–3. An instance is called *applicative* for a path must meet the following two conditions:

- Each task in a path can complete before its latest finish time when the path is scheduled on the instance.
- The increased cost of the new schedule, including computation and data transfer, is less than the cost of initiating any new instance to schedule the path. It means that the new schedule should use (a part of) idle time slot between two already-scheduled tasks or the extra time, which is a surplus of the last time interval, on the instance.

There are three factors that need to be considered for finding an *applicative* instance.

- Because the same instance executes the whole tasks of the path, the data transfer time between each pair of tasks on the path changes to be 0, but the data transfer time from external tasks should be taken into account. If the parent (child) of a task belong to the path is executed on an existing instance, the data transfer time between them

TABLE II
THE EXECUTION TIME OF THE TASKS ON EACH COMPUTATION SERVICE

		t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}	t_{11}	t_{12}
C_1	s_{11}	2	5	3	7	3	2	1	6	5	1	4	13
	s_{12}	10	31	28	60	26	21	5	43	41	10	33	103
C_2	s_{21}	5	8	7	12	5	4	2	13	8	3	9	25
	s_{22}	9	17	11	20	8	10	3	21	13	4	14	41

changes to be 0 when the path is scheduled on the same instance. Also, if the parent (child) of a task belonging to the path is executed on an existing instance, the data transfer time between them changes to be $TT_{intra}(e_{ij}, s_{pi}, s_{pj})$ when the path is scheduled on the other instance offered by the same provider. Finally, if the parent (child) of a task belonging to the path is executed on an existing instance, the data transfer time between them changes to be $TT_{inter}(e_{ij}, s_{pi}, s_{qj})$ and the data transfer cost should be considered when the path is scheduled on the other provider.

- The computation part of the increased cost for employing the surplus time of an existing instance is 0.
- The algorithm considers each idle time-slot between two scheduled tasks to schedule the path with its updated *EST* and *LFT* on an existing instance. Otherwise, the algorithm tries to schedule the path after the last task or before the first task of the instance.

If an *applicative* existing instance with the minimum cost growth does not exist, a new instance from the cheapest service will be initiated to execute each task of the PCP before its *LFT* in lines 4–6. After the PCP is scheduled on the 'best-fit instance' in line 7, some parameters related to each task of the PCP such as selected service, actual start time and end time, as well as scheduling status are set in lines 8–12.

C. Illustrative Example

In order to illustrate the execution of our proposed algorithm, we trace its detailed operations on the workflow sample shown in Fig. 2. The number above each arc represents the amount of transfer data between two corresponding tasks. We assume that the data transfer cost between two different providers is 0. There are two cloud providers (i.e., C_1 and C_2), and each provider has two computation services (i.e., s_{11} and s_{12} in C_1 , s_{21} and s_{22} in C_2). The execution time of the tasks on each computation service (i.e., instance) is shown in Table II. The intra-bandwidth and inter-bandwidth are assumed to be 2 and 1, respectively, and the boot time of a new leased instance is set to be 0. Each computation service' paying time interval is assumed to be 5, and the execution cost per time interval is set to be 8 for s_{11} , 1 for s_{12} , 4 for s_{21} , and 2 for s_{22} . Finally, the workflow's deadline is assumed to be 60.

When the MCPCPP algorithm is called for the workflow sample shown in Fig. 2, it first called the *preProcess* procedure for merging the adjacent two tasks with a common 'directed cut-edge' into a single task iteratively. The *preProcess* procedure (Algorithm II) finds two 'directed cut-edges', namely the

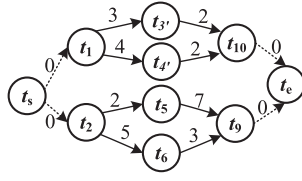


Fig. 5. A new workflow with different structure.

TABLE III
THE VALUE OF *EFT*, *EST*, *LST* AND *LFT* FOR EACH STEP OF RUNNING MCPCPP

		t_1	t_2	t_3'	t_4'	t_5	t_6	t_7	t_{10}
Ori	<i>EFT</i>	2	5	7.5	17	9	9.5	17.5	19
	<i>EST</i>	0	0	3.5	4	6	7.5	12.5	18
	<i>LST</i>	41	42.5	54	45	48.5	51.5	55	59
	<i>LFT</i>	43	47.5	58	58	51.5	53.5	60	60
St_1	<i>EFT</i>	9*	5	14.5*	50*	9	9.5	17.5	54*
	<i>EST</i>	0*	0	10.5*	9*	6	7.5	12.5	50*
	<i>LST</i>	41	42.5	45*	45	48.5	51.5	55	59
	<i>LFT</i>	43	47.5	49*	56	51.5	53.5	60	60
$St_{1.1}$	<i>EFT</i>	9	5	45*	50	9	9.5	17.5	54
	<i>EST</i>	0	0	12*	9	6	7.5	12.5	50
	<i>LST</i>	41	42.5	45	45	48.5	51.5	55	59
	<i>LFT</i>	43	47.5	49	56	51.5	53.5	60	60
St_2	<i>EFT</i>	9	17*	45	50	21*	27*	40*	54
	<i>EST</i>	0	0*	12	9	18*	17*	27*	50
	<i>LST</i>	41	42.5	45	45	21*	51.5	55	59
	<i>LFT</i>	43	47.5	49	56	23.5*	53.5	60	60
$St_{2.1}$	<i>EFT</i>	9	17	45	50	23*	27	40	54
	<i>EST</i>	0	0	12	9	18*	17	27	50
	<i>LST</i>	41	42.5	45	45	21	51.5	55	59
	<i>LFT</i>	43	47.5	49	56	23.5	53.5	60	60
Instance		$s_{2,2,1}$	$s_{2,2,2}$	$s_{1,2,1}$	$s_{2,2,1}$	$s_{2,1,1}$	$s_{2,2,2}$	$s_{2,2,2}$	$s_{2,2,1}$

arc from t_3 to t_7 and the arc from t_4 to t_8 . It merges them into a single task iteratively, and generates a new workflow with different structure shown in Fig. 5. The execution time on different computation services for the new merged tasks, namely t_3' and t_4' , are shown as the last two columns in Table II. Then the MCPCPP algorithm computes $EFT(t_i)$, $EST(t_i)$, $LST(t_i)$ and $LFT(t_i)$ for each task in the new workflow. These parameters' original value are shown as the *Ori* row in Table III. Finally, it initializes some parameters and calls the *Schedule_all_Parents* procedure with the input t_e .

The pseudo exit-task t_e has two precursors, so the while loop in line 1 in Algorithm III will executes two times, which are represented as step 1 (St_1) and step 2 (St_2). The updated values of *EFT*, *EST*, *LST*, and *LFT* for each scheduling procedure are given in Table III. Besides, the star mark '*' is used to highlight that the value has been changed after the previous step. In order to save space, we indicate the value of *AST* and *AET* in the *EST* and *EFT* rows, respectively, after the task is scheduled. Therefore, the *EST* and *EFT* rows show the value of *AST* and *AET* for the scheduled tasks.

St_1 : The *Schedule_all_Parents* procedure first finds the t_e 's partial critical path, which is $t_1 - t_4' - t_{10}$. Then the *Schedule_Path* procedure (Algorithm IV) is called to schedule these tasks. It is the first time to lease the instances, so the algorithm launches a new instance $s_{2,2,1}$, which is the cheapest

computation service that can finish each task before their *LFTs*. Note that the data transfer time between these tasks is 0. After the PCP is scheduled, the *LSTs* and *LFTs* of these tasks' predecessors, i.e., t_3' , and the *EFTs* and *ESTs* of their successors, i.e., t_7 , are updated in St_1 row. Finally, it calls the *Schedule_all_Parents* procedure for all the tasks on the PCP. Due to the t_1 and t_4' have no unscheduled parents, the *Schedule_all_Parents* procedure with the input t_{10} is called in $St_{1.1}$.

$St_{1.1}$: It first finds the t_{10} 's partial critical path, namely t_3' . Then it calls the *Schedule_Path* procedure to schedule t_3' with launching a new instance $s_{1,2,1}$. Since the PCP $t_1 - t_4' - t_{10}$ and t_3' are not scheduled in the same cloud, we should consider the inter-bandwidth between t_1 and t_3' . Finally, there is no unscheduled predecessor or successor of t_3' , the step St_1 finishes here.

St_2 : Now, back to the pseudo exit-task t_e , the *Schedule_all_Parents* procedure finds the last partial critical path of t_e , which is $t_2 - t_6 - t_9$. Then it calls the *Schedule_Path* procedure for scheduling these tasks on a new instance $s_{2,2,2}$, which is the cheapest instance that can finish these tasks before their *LFTs*. The new values the *LSTs* and *LFTs* of these tasks' predecessors, i.e., t_5 , and the *EFTs* and *ESTs* of their successors, i.e., t_5 , are updated. Finally, the *Schedule_all_Parents* procedure is called for all the tasks on the PCP. The task t_2 and t_6 have no unscheduled parents, so we just think over t_9 in $St_{2.1}$.

$St_{2.1}$: It first finds the partial critical path of t_9 , which is t_5 . Then the *Schedule_Path* procedure is called for scheduling t_5 with launching a cheapest instance $s_{2,1,1}$. Since there is no unscheduled parent or child of t_5 , the algorithm stops here.

The mapping from each task to the corresponding instances is shown as the last row in Table III. Therefore, the total execution cost is 49, and the total execution time equals 54.

V. PERFORMANCE EVALUATION

All the simulated tests were conducted on 64-bit Win7, which was configured with Intel i7-3610QM cores, 2.30 GHz frequency CPU and 8 GB memory. In this section, we presented the experimental setup, the competitive algorithms compared with MCPCPP and the performance of our scheduling strategy.

A. Experimental Setup

In order to evaluate the Big Data workflow scheduling strategy, the best choice is to measure its performance through a library of real-world workflows acquired from scientific community. Unfortunately, there is no such a library available for the scientific research, so we have to construct the workflows with the help of partly synthetic model. Bharathi et al. [34] did the primary works and investigated the structure of five real-world workflows from different scientific areas: Montage from astronomy, CyberShake from earthquake science, Epigenomics from biology, LIGO from gravitational physics and SIPHT from biology. They also discussed a detailed computation and data requirements for each workflow. The approximate structure of each workflow is shown in Fig. 6, which has different

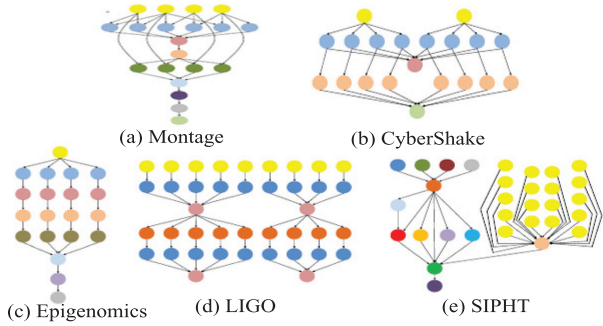


Fig. 6. The structure of five real-world Big Data workflows.

TABLE IV
THE AVAILABLE BANDWIDTH BETWEEN TWO DIFFERENT PROVIDERS

	Avg (MB/s)	Time/GB(s)	Stdev(%)
C1-C2	1.33	770	29.50
C1-C3	3.25	315	18.23
C2-C3	5.43	189	33.26

characteristics in terms of their basic components and composition. Furthermore, they developed a workflow generator to create synthetic workflows, which were similar to the real-world Big Data workflows. There are four different numbers of tasks for each scientific workflow created from the generator in the XML format¹, from which we selected three types of workflow for our experiments: Small (about 30 tasks), Medium (about 100 tasks) and Large (about 1000 tasks).

We assumed that there were three providers (i.e., C1, C2 and C3) in the Multi-cloud environment. Each cloud provider offered 8 computation services with different processing speeds and prices per hour. The processor speeds of the available services were set randomly such that the slowest service was approximately 5, 8 and 10 times slower than the fastest one in provider C1, C2 and C3, respectively. Accordingly, the slowest one was roughly 5, 8 and 10 times cheaper the fastest one in provider C1, C2 and C3, respectively. Each task of a workflow had different price/performance ratio in different clouds [30] and we assumed that each one-third of tasks had better price/performance ratio in one cloud compared with the other two clouds. The average bandwidths between two different instances inside a single cloud provider were set to be 20 MBps, which were the approximate bandwidths between two different Amazon computation services [35]. Also, the average bandwidths between two different clouds were set as Table IV, which were measured roughly with the help of bandwidth measurement tool iperf² for Sydney region of Amazon EC2 (C1), Rackspace (C2) and GoGrid (C3). In addition, we adopted an instance boot time of 97 seconds and an instance charge interval of 1 hour, which were selected based on the results for Amazon EC2 [8]. Table V describes the data transfer prices for Amazon EC2 (C1), Rackspace (C2) and GoGrid (C3), which were derived from their corresponding official website.

Finally, in order to test our scheduling strategy, each workload requires a deadline. Setting a deadline too short will result

TABLE V
THE DATA TRANSFER COST OF DIFFERENT PROVIDER

	Price/GB(\$)			
	0-1GB	1GB-1TB	1TB-10TB	10TB-50TB
C1→(C2,C3)	0.00	0.19	0.19	0.15
C2→(C1,C3)	0.12	0.12	0.12	0.10
C3→(C2,C3)	0.00	0.12	0.11	0.10

in the rejection of the workflow since no resource can be found to meet the deadline, so we defined nine different deadlines for each Big Data workflow W as formula (19):

$$D_i(w) = r_i \cdot \text{Min}(w), i = \{1, \dots, 9\} \quad (19)$$

where $\text{Min}(w)$ is the makespan of executing workflow W with HEFT [36] strategy, and r_i is selected in turn from the set of $R = 1.2, 1.5, 2, 3, 5, 8, 10, 15, 20$. With the definition of deadline, we could guarantee each workload would be completed successfully.

B. The Competitive Algorithms

To our best knowledge, there were no similar algorithms proposed to address our proposed problem. In order to test our proposed scheduling algorithm, we adapted the original PCP [28] and PSO-based [5] algorithms to the Multi-cloud environments. As mentioned in Section II, the original PCP workflow scheduling algorithm had the similar criterion, namely minimizing the workflow's execution cost while meeting its deadline. It included two main phases: *Deadline Distribution* and *Planning*. A specific deadline for a workflow was distributed over all the individual tasks in the *Deadline Distribution* phase. Firstly, the overall actual critical path of the workflow from the exit task was found, and the *Path Assigning* procedure with three different policies was called to distribute the deadline among the critical tasks. Each critical task had a specific sub-deadline for all of its predecessors after the distribution. Secondly, the same procedure was called from all the critical tasks with their sub-deadlines as deadlines, and then the *Path Assigning* procedures were called, respectively. This process continued recursively until all the tasks in the workflow were assigned with a sub-deadline. Finally, *Planning* phase scheduled each task to its corresponding service with the cheapest cost while meeting its sub-deadline.

We compared our proposed MCPCPP algorithm with Multi-cloud-Original-PCP (MOP), which was a simple transformation of the original PCP in Multi-clouds. In order to adapt to the new price model and the Multi-cloud environment, MOP maintained a similar two-phase (*Deadline Distribution* and *Planning*) structure to the original PCP but with two main distinctions. First, MOP had only one new fair *Path Assigning* policy instead of three different policies in PCP. The *Path Assigning* procedure distributed the path sub-deadline fairly among each task t_i of the path in proportion to its $MET(t_i)$. The path sub-deadline, PSD , is defined as formula (20), which was the difference between the $LFT(t_l)$ of the last task t_l and the $EST(t_f)$ of the first task t_f on the path. The sub-deadline

¹<https://confluence.pegasus.isi.edu/display/pegasus/WorkflowGenerator>.

²<http://iperf.sourceforge.net/>.

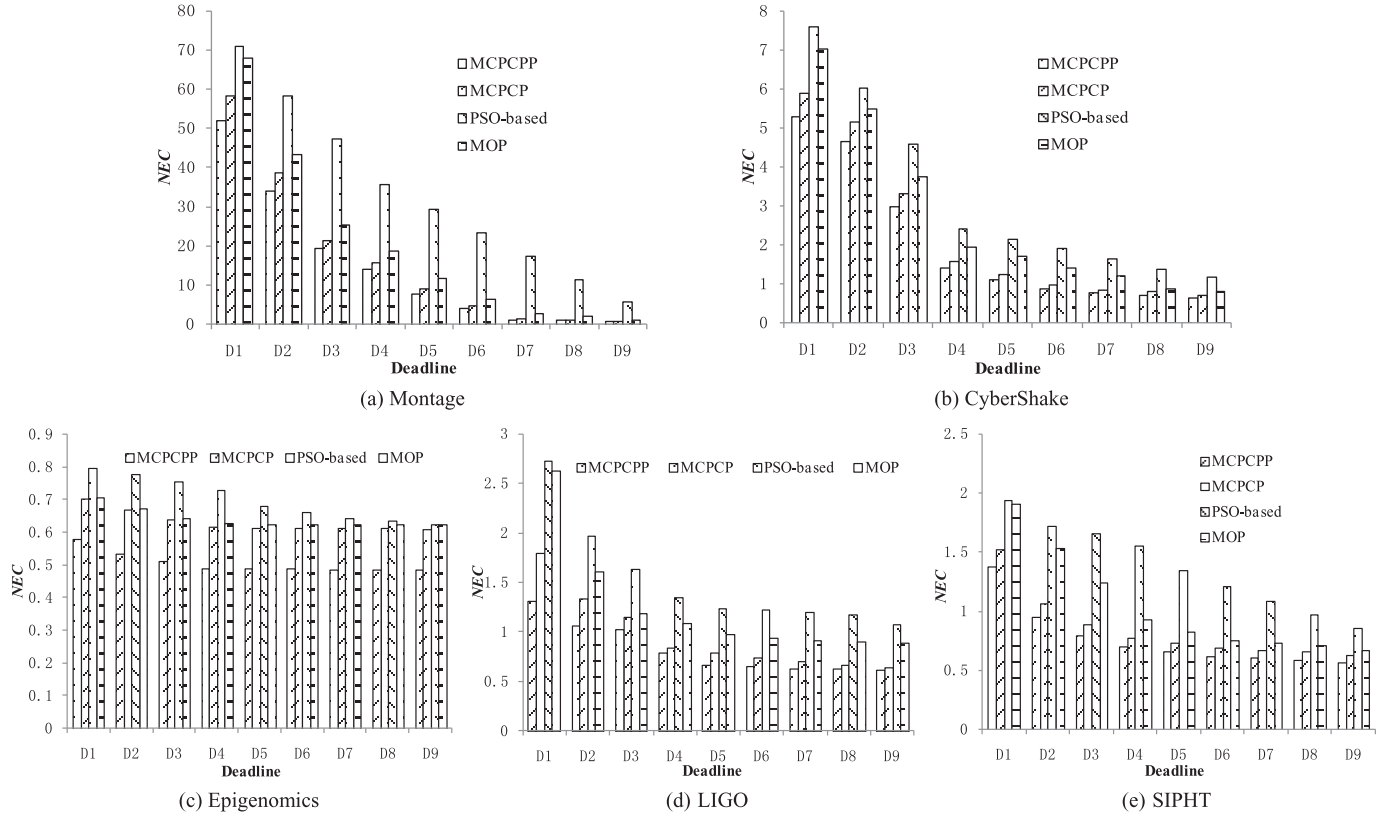


Fig. 7. The normalized execution cost of five large size real-world workflows with four different scheduling algorithms in Multi-clouds.

of each task t_i on the path, $SD(t_i)$, is shown in formula (21):

$$PSD = LFT(t_i) - EST(t_f) \quad (20)$$

$$SD(t_i) = EST(t_f) + PSD \cdot \frac{EFT(t_i) - EST(t_f)}{EFT(t_i) - EST(t_f)} \quad (21)$$

Second, the *Planning* phase was modified to schedule a ready-task on a ‘best-fit instance’, whose definition for a task with its sub-deadline was similar to the definition for a PCP in Section IV.

PSO-based algorithm on the other hand, was also a static algorithm pursuing the same objectives in a single cloud. However, it could be adjusted to schedule a Big Data workflow in Multi-clouds with less modification. The PSO-based algorithm first determined the maximum number of tasks that could execute in parallel for a given workflow. This was done in order to decrease the initial resource pool and reduce the search space size of PSO-based algorithm. In our comparative experiments, the initial resource pool contained all the instance types from all cloud providers. After this, it generated the PSO model including problem encoding and goodness defining two key steps. Problem encoding was similar to the traditional one, in which a particle’s position encoded a mapping of tasks to resources. Goodness defining introduced the constraint-handling strategy [37] for a better solution when there were two available solutions. Finally, the workflow schedule converted from each particle’s position was generated. This procedure considered both the instance booting time and data transfer time when the adjacent tasks were scheduled in different instances. Moreover, we took the data transfer time and cost

from different cloud providers into account. In the comparative experiments, the large size workflows were used and the other relevant parameters were the same as in [5].

In order to test the improvement of MCPCPP compared with our previous work [15], we chose the MCPCP as another comparative algorithm. MCPCP was designed for the same problem as discussed in Section III, namely scheduling a Big Data workflow in Multi-clouds while minimizing the total execution cost and satisfying its deadline. It also considered computation time of each task in addition to data transfer time. Moreover, the algorithm also adapted the PCPs to reduce the data transfer time between two adjacent tasks.

C. Performance Results

Since there are many different kinds of workloads with various properties, it is necessary to normalize the execution cost for each workload. We defined the normalized execution cost of a workflow w , $NEC(w)$, as formula (22):

$$NEC(w) = \frac{SC(w)}{CC(w)} \quad (22)$$

where $SC(w)$ is the total execution cost of the workflow W in Multi-clouds with a scheduling strategy, and $CC(w)$ represents the cost of the *cheapest scheduling*, which executes all the tasks in a single instance of the cheapest computation service from one of the cloud providers, i.e., the cheapest cost in a single-cloud scheduling, based on their precedence constraints.

Fig. 7 shows the normalized execution cost of five large size workflows with four different scheduling algorithms (i.e.,

TABLE VI
THE PRE AND POST TASK AMOUNT WITH PRETREATMENT OF MCPCPP
ALGORITHM

Size	State	CyberShake	Epigenomics	LIGO	Montage	SIPHT
Small	Post	30	7	16	21	29
	Pre	30	24	30	25	29
Medium	Post	100	26	53	96	97
	Pre	100	100	100	100	97
Large	Post	1000	260	514	996	968
	Pre	1000	997	1000	1000	968

MCPCPP, MCPCP, PSO-based and MOP) in Multi-clouds. Please note that all cases are successful with different assigned deadlines, even with the tightest deadlines, i.e., $D_1(w)$. With a quick view on the five subgraphs of Fig. 7, the $NEC(w)$ of each algorithm decreases with the increment of the assigned deadlines. In particular, the PSO-based' $NEC(w)$ has a slower declining rate as the deadline increased, and its gap between the loose deadline and tight deadline is the smallest among different algorithms. This is mainly due to the global random optimization technique, which ignores the data transfer optimization between each pair of connected tasks. Moreover, the traditional particle updated policy of PSO-based is not sensitive to the changes of deadline. MCPCPP has a better performance than other algorithm in all cases because of the local novel data transfer compression and pretreatment strategies. On the other hand, PSO-based has the worst performance because it does not consider the internal structure of workflow. In some case of our experiments, such as CyberShake, Epigenomics and SIPHT workflows, the $NEC(w)$ of all algorithms but PSO-based are less than 1 with a loose deadline, which implies that scheduling a Big Data workflow in Multi-clouds can perform better than in a single cloud in some cases, even with a slower inter-bandwidth.

With the structure influence of different workflows, MCPCPP observably outperforms MOP for the LIGO and SIPHT workflows, and it is slightly better for the Montage and CyberShake, and it has an approximately similar performance in the case of Epigenomics. With the help of pretreatment procedure and 'best-fit instance' utilization, MCPCPP performs better than MCPCP in all cases. The number of merged 'directed cut-edges' has a significant effect on the $NEC(w)$ of MCPCPP. For instance, the average $NEC(w)$ improvement of MCPCPP is about 26% compared with MCPCP for the Epigenomics, which has less than 30 percentage of its original task amount after pretreatment procedure as shown in Table VI. On the other hand, MCPCPP performs slightly better than MCPCP for the CyberShake and SIPHT because there are no merge operators as shown in Fig. 7(b) and (e).

The $NEC(w)$ of each algorithm is very high for the Montage workflow with a tight deadline in Fig. 7(a), which is about 10 times more expensive compared with the CyberShake. The reason for the $NEC(w)$ gap is the different structures among various workflows. Fig. 6(a) shows the rough structure of the Montage, and we can find that there are many tasks in the

second row. Moreover, each task of the Montage in the second row will cost about 15 seconds execution time on the fastest computation service. When the assigned deadline is tight, the scheduler has to assign them with the distinct fastest instances, which can guarantee that each task in the second row will be finished before the latest start time of their directed successors, i.e., the single task in the third row. Therefore, more instances of the computation service will be launched for the Montage workflow with a tight deadline, and only a small fraction of their paying time intervals is utilized. In our experiments, it is only about 2% utilization rate for the instances to schedule the second row's tasks of Montage with $D_1(w)$. The CyberShake workflow has a similar structure about the tasks in the second row as shown in Fig. 6(b). However, the Fig. 7(b) shows that the $NEC(w)$ of CyberShake for each algorithm is less compared with Montage with a tight deadline. There are three main reasons for generating the difference between the CyberShake and Montage workflows. Firstly, the percentage of the second row's tasks in a workflow is different. The percentage for CyberShake is about 41% and the percentage for Montage is about 56%. Obviously, more percentage of the second row's tasks in a workflow with a tight deadline will generate more under-utilized instances resulting in a higher cost. Secondly, the execution time of the second row's tasks on the fastest computation services is different. It costs about 800 seconds for CyberShake and about 15 seconds for Montage, respectively. The execution times approaching to an integrated time interval will reduce the overhead of paying time intervals, which causes the overall cost of scheduling the CyberShake workflow notably less than the Montage workflow. Finally, the structure of the third row is different. There are many tasks in the third row of the CyberShake. However, the Montage has only one task in its third row, which will result in that each task in the second row has to be scheduled independently on the faster instance with a tighter deadline. As a result, it will take more cost to execute the Montage workflow compared with CyberShake.

The general structure of the experimental results for the medium and small size workflows with different algorithms is similar to the large size workflows. The main difference among the results of the various workflow sizes is the value of $NEC(w)$. Due to the space limitation, we do not discuss the differences in details but upload the complete experimental results to the Internet³.

Table VI shows the pre and post task amount with the pretreatment procedure of MCPCPP algorithm. The Epigenomics and LIGO workflows lose about 70% and 50% task amount, respectively, and the Montage workflow loses a little after pretreatment. The compression rate has a close connection with the performance of MCPCPP, which can be interpreted through combining with the subgraphs of Fig. 7.

Finally, the real runtime of four different scheduling algorithms with different type and size workflows is shown in Table VII, which is measured in millisecond (ms). Due to the fluctuation of our experimental environment, we acquire each algorithm's runtime with the average of 100 repeated experiments. In some cases, the runtime of MCPCPP is better than

³<http://yunpan.cn/cQu3thitXWnUC>. Password: 6628

TABLE VII
THE REAL RUNTIME OF FOUR SCHEDULING ALGORITHMS WITH
DIFFERENT SIZE WORKFLOWS (MS)

Size	Algorithm	CyberShake	Epigenomics	LIGO	Montage	SIPHT
Small	MCPCPP	6	2	5	5	7
	MCPCP	6	4	6	5	7
	PSO-based	281	234	239	224	208
	MOP	3	2	3	2	3
	MCPCPP	25	8	18	37	28
Medium	MCPCP	24	15	23	37	27
	PSO-based	425	302	373	393	338
	MOP	13	10	12	20	12
Large	MCPCPP	862	241	705	1414	721
	MCPCP	860	392	904	1345	708
	PSO-based	1626	2019	1563	1193	1781
	MOP	684	307	539	731	382

MCPCP, such as for the Epigenomics and LIGO workflows. This is due to the pretreatment procedure, which compresses the amount of tasks to reduce the runtime of scheduling tasks. However, the pretreatment procedure will also cost extra runtime if there is no ‘directed cut-edge’ in a workflow, such as CyberShake and SIPHT. Therefore, it will cost more runtime for such workflows with MCPCPP algorithm. MOP has the best runtime in almost all cases, since it does not consider the task precedence and sub-deadline update after the ‘deadline distribution’ procedure. PSO-based is a global random optimization algorithm. The runtime of PSO-based is mainly dependent on the number of articles and the maximum iterative times. Based on the configured parameters, POS-based algorithm requires more runtime to get a global best result, especially for the small size workflows. In our experiments, we find that larger task amount does not necessarily mean more runtime for PSO-based algorithm. For example, the CyberShake workflow with 50 tasks only takes 246.11 ms, but it takes 280.78 ms to run the small size workflow as shown in Table VII.

VI. CONCLUSION

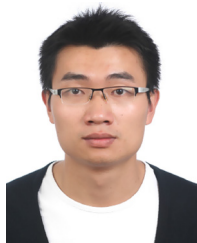
In this paper, we propose a novel cost-driven workflow scheduling strategy, namely MCPCPP for Big Data applications in Multi-clouds. The approach utilizes the main function of PCP and aims to minimize the execution cost of workflow while meeting the deadline requirement. MCPCPP takes the essential characteristics of the Multi-cloud environment into account. Moreover, it considers not only the computation cost but also the data transfer cost. Various well-known Big Data workflows are used to evaluate our strategy and the experimental results show that MCPCPP has a better performance than many other comparative algorithms in all different cases. It also shows that the pretreatment procedure has a significant effect on the performance and runtime of MCPCPP. From the $NEC(w)$ results, we observe that scheduling a workflow in Multi-clouds can perform better than in a single cloud under a loose deadline due to more choices of resources.

In the future, we aim to explore the influence of inaccurate execution time and data transfer time for a Big Data workflow in Multi-clouds. We will further refine our strategy to accommodate the fluctuating environment. Another future work will be on extending the infrastructure model including private and community clouds, which will make our problem more challenging. Finally, we plan to implement our scheduling strategy in workflow engine for deploying Big Data applications in real-world environment.

REFERENCES

- [1] S. Sagioglu and D. Sinanc, “Big data: A review,” in *Proc. Int. Conf. Collab. Technol. Syst. (CTS)*, 2013, pp. 42–47.
- [2] J. Yin *et al.*, “BURSE: A bursty and self-similar workload generator for cloud computing,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 26, no. 3, pp. 668–680, Mar. 2015.
- [3] N. Xiong *et al.*, “A distributed efficient flow control scheme for multirate multicast networks,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 21, no. 9, pp. 1254–1266, Sep. 2010.
- [4] B. Lin, W. Guo, and X. Lin, “Online optimization scheduling for scientific workflows with deadline constraint on hybrid clouds,” *Concurrency and Computation: Practice and Experience*, 2015, doi: 10.1002/cpe.3582.
- [5] M. A. Rodriguez and R. Buyya, “Deadline based resource provisioning and scheduling algorithm for scientific workflows on clouds,” *IEEE Trans. Cloud Comput.*, vol. 2, no. 2, pp. 222–235, Apr./Jun. 2014.
- [6] Y. Ren *et al.*, “Mutual verifiable provable data auditing in public cloud storage,” *J. Internet Technol.*, vol. 16, no. 2, pp. 317–323, 2015.
- [7] J. Jofre *et al.*, “Federation of the BonFIRE multi-cloud infrastructure with networking facilities,” *Comput. Netw.*, vol. 61, pp. 184–196, 2014.
- [8] J. Yin *et al.*, “JTangCSB: A cloud service bus for cloud and enterprise application integration,” *IEEE Internet Comput.*, vol. 19, no. 1, pp. 35–43, Jan./Feb. 2015.
- [9] V. Varadharajan and U. Tupakula, “Security as a service model for cloud environment,” *IEEE Trans. Netw. Service Manage.*, vol. 11, no. 1, pp. 60–75, Mar. 2014.
- [10] E. Deelman, “Grids and clouds: Making workflow applications work in heterogeneous distributed environments,” *Int. J. High Perform. Comput. Appl.*, vol. 24, no. 3, pp. 284–298, 2010.
- [11] Y. K. Kwok and L. Ahmad, “Dynamic critical-path scheduling: An effective technique for allocating task graphs to multiprocessors,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 7, no. 5, pp. 506–521, May 1996.
- [12] Z. Pan, Y. Zhang, and S. Kwong, “Efficient motion and disparity estimation optimization for low complexity multiview video coding,” *IEEE Trans. Broadcast.*, vol. 61, no. 2, pp. 166–176, Jun. 2015.
- [13] S. F. Chang and D. G. Messerschmitt, “Designing high-throughput VLC decoder. I. Concurrent VLSI architectures,” *IEEE Trans. Circuits Syst. Video Technol.*, vol. 2, no. 2, pp. 187–196, Jun. 1992.
- [14] N. Xiong *et al.*, “Comparative analysis of quality of service and memory usage for adaptive failure detectors in healthcare systems,” *IEEE J. Selected Areas in Communications*, vol. 27, no. 4, pp. 495–509, May 2009.
- [15] B. Lin *et al.*, “Cost-driven scheduling for deadline-constrained workflow on multi-clouds,” in *Proc. Parallel Distrib. Process. Symp. Workshop (IPDPSW)*, 2015, pp. 1191–1198.
- [16] T. Sousa, A. Silva, and A. Neves, “Particle swarm based data mining algorithms for classification tasks,” *Parallel Comput.*, vol. 30, no. 5, pp. 767–783, 2004.
- [17] W. N. Chen and J. Zhang, “An ant colony optimization approach to a grid workflow scheduling problem with various QoS requirements,” *IEEE Trans. Syst., Man, Cybern. C, Appl. Rev.*, vol. 39, no. 1, pp. 29–43, Jan. 2009.
- [18] A. Hiraes-Carbajal *et al.*, “Multiple workflow scheduling strategies with user run time estimates on a grid,” *J. Grid Comput.*, vol. 10, no. 2, pp. 325–346, 2012.
- [19] H. Cao *et al.*, “DAGMap: Efficient and dependable scheduling of DAG workflow job in Grid,” *J. Supercomput.*, vol. 51, no. 2, pp. 201–223, 2010.
- [20] J. Yu and R. Buyya, “A budget constrained scheduling of workflow applications on utility grids using genetic algorithms,” in *Proc. Workflows Support Large-Scale Sci. Workshop (WORKS)*, 2006, pp. 1–10.

- [21] S. Abrishami, M. Naghibzadeh, and D. H. Epema, "Cost-driven scheduling of grid workflows using partial critical paths," *IEEE Trans. Parallel Distrib. Syst.*, vol. 23, no. 8, pp. 1400–1414, Aug. 2012.
- [22] M. Malawski *et al.*, "Cost-and deadline-constrained provisioning for scientific workflow ensembles in iaas clouds," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2012, p. 22.
- [23] M. Mao and M. Humphrey, "Auto-scaling to minimize cost and meet application deadlines in cloud workflows," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2011, p. 49.
- [24] K. Keahey *et al.*, "Sky computing," *IEEE Internet Computing*, vol. 13, no. 5, pp. 43–51, Sep./Oct. 2009.
- [25] R. Buyya, R. Ranjan, and R. N. Calheiros, "Intercloud: Utility-oriented federation of cloud computing environments for scaling of application services," in *Algorithms and Architectures for Parallel Processing*, New York, NY, USA: Springer, 2010, pp. 13–31.
- [26] H. M. Fard, R. Prodan, and T. Fahringer, "A truthful dynamic workflow scheduling mechanism for commercial multicloud environments," *IEEE Trans. Parallel Distrib. Syst.*, vol. 24, no. 6, pp. 1203–1212, Jun. 2013.
- [27] J. Li *et al.*, "Online optimization for scheduling preemptable tasks in IaaS cloud systems," *J. Parallel Distrib. Comput.*, vol. 72, no. 5, pp. 666–677, 2012.
- [28] S. Abrishami, M. Naghibzadeh, and D. H. Epema, "Deadline-constrained workflow scheduling algorithms for infrastructure as a service clouds," *Future Gener. Comput. Syst.*, vol. 29, no. 1, pp. 158–169, 2013.
- [29] S. S. Woo and J. Mirkovic, "Optimal application allocation on multiple public clouds," *Comput. Netw.*, vol. 68, pp. 138–148, 2014.
- [30] L. F. Bittencourt, E. R. Madeira, and N. L. Da Fonseca, "Scheduling in hybrid clouds," *IEEE Communications Magazine*, vol. 50, no. 9, pp. 42–47, Sep. 2012.
- [31] Z. Xia *et al.*, "A secure and dynamic multi-keyword ranked search scheme over encrypted cloud data," *IEEE Trans. Parallel Distrib. Syst.*, vol. 27, no. 2, pp. 340–352, Feb. 2016.
- [32] K. Dinkla, M. A. Westenberg, and J. J. van Wijk, "Compressed adjacency matrices: Untangling gene regulatory networks," *IEEE Trans. Vis. Comput. Graphics*, vol. 18, no. 12, pp. 2457–2466, Dec. 2012.
- [33] F. Zhao *et al.*, "The impact of non-cut link failures on network traffic," *J. Commun.*, vol. 27, no. z1, pp. 184–188, 2006.
- [34] S. Bharathi *et al.*, "Characterization of scientific workflows," in *Proc. Workflows Support Large-Scale Sci. Workshop (WORKS)*, 2008, pp. 1–10.
- [35] Z. Fu *et al.*, "Achieving efficient cloud search services: Multi-keyword ranked search over encrypted cloud data supporting parallel computing," *IEICE Trans. Commun.*, vol. E98.B, no. 1, pp. 190–200, 2015.
- [36] H. Topcuoglu, S. Hariri, and M. Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 3, pp. 260–274, Mar. 2002.
- [37] C. Ge *et al.*, "Energy management in cross-domain content delivery networks: A theoretical perspective," *IEEE Trans. Netw. Service Manage.*, vol. 11, no. 3, pp. 264–277, Sep. 2014.



Bing Lin received the bachelor's degree in computer science from Fuzhou University, Fuzhou, China, in 2010. He is currently pursuing the Ph.D. degree at the College of Communication and Information System, Fuzhou University. His research interests include parallel and distributed computing, computational intelligence, and data center resource management. He is now undertaking the research work on reducing the execution cost of workflow in cloud with the constrained optimization.



Wenzhong Guo received the B.S. and M.S. degrees in computer science from Fuzhou University, Fuzhou, China, in 2000 and 2003, respectively, and the Ph.D. degree in communication and information system from Fuzhou University in 2010. He is currently a Full Professor with the College of Mathematics and Computer Science, Fuzhou University. His research interests include cloud computing, mobile computing, and evolutionary computation. Currently, he leads the Network Computing and Intelligent Information Processing Laboratory, which is a Key Laboratory of Fujian Province, China.



Naixue Xiong (SM'12) received the Ph.D. degree in software engineering from Wuhan University, Wuhan, China, and the Ph.D. degree in dependable networks from Japan Advanced Institute of Science and Technology, Nomi, Japan. He is currently a Faculty Member with the Department of Business and Computer Science (BCS), Southwestern Oklahoma State University (SWOSU), Weatherford, OK, USA. Before he joined SWOSU, he worked with Colorado Technical University, Colorado Springs, CO, USA (Full Professor, 4 years), Wentworth Technology Institution, Boston, MA, USA (Assistant Professor, one year), and Georgia State University, Atlanta, GA, USA (Assistant Professor, Post-Doc, 4 years). His research interests include cloud computing, business networks, security and dependability, parallel and distributed computing, and optimization theory. He authored more than 100 international journal papers and more than 100 international conference papers. Some of his work has been published in the IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS, IEEE/ACM TRANSACTIONS, ACM Sigcomm Workshop, IEEE INFOCOM, ICDCS, and IPDPS. He has been the General Chair, the Program Chair, the Publicity Chair, a PC member, and an OC member of more than 100 international conferences. He served as the Editor-in-Chief, an Associate Editor, or an Editor member for more than 10 international journals (including as Associate Editor for the IEEE TRANSACTIONS ON SYSTEMS, MAN, AND CYBERNETICS: SYSTEMS, Associate Editor for *Information Science*, Editor-in-Chief for the *Journal of Internet Technology*, and Editor-in-Chief for *Journal of Parallel and Cloud Computing*), and Guest Editor for more than 10 international journals, including IEEE WIRELESS COMMUNICATIONS, *Sensor Journal*, *WINET*, and *MONET*. He was the recipient of the Best Paper Award in the 10th IEEE International Conference on High Performance Computing and Communications (HPCC-08) and the Best Student Paper Award in the 28th North American Fuzzy Information Processing Society Annual Conference (NAFIPS2009).



Guolong Chen received the B.S. and M.S. degrees in computational mathematics from Fuzhou University, Fuzhou, China, in 1987 and 1992, respectively, and the Ph.D. degree in computer science from Xi'an Jiaotong University, Xi'an, China, in 2002. He is a Full Professor with the College of Mathematics and Computer Science, Fuzhou University. His research interests include cloud computing, computation intelligence, computer networks, and information security.



Athanasios V. Vasilakos is currently a Professor with the Luleå University of Technology, Luleå, Sweden. He has served as an Editor for many technical journals such as the IEEE TRANSACTIONS ON NETWORK AND SERVICE MANAGEMENT, IEEE TRANSACTIONS ON CLOUD COMPUTING, IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, IEEE TRANSACTIONS ON CYBERNETICS, IEEE TRANSACTIONS ON NANOBIOSCIENCE, IEEE TRANSACTIONS ON INFORMATION TECHNOLOGY IN BIOMEDICINE, *ACM Transactions on Autonomous and Adaptive Systems*, and IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS. He is also the General Chair of the European Alliances for Innovation.



Hong Zhang received the B.S. degree with the Department of Mathematics and Computer Science, Fuzhou University, Fuzhou, China, in 2013. He is currently pursuing the master's degree at the College of Mathematics and Computer Science, Fuzhou University. His research interests include distributed file systems, cloud computing, and cloud storage system. He is now undertaking the research work on improving the performance of the data replica policy in cloud storage system.